

**Course: Fundamentals of Generative AI and Large Language
Models: Theory and Practice**

**Week 2: Autoencoders: The Representation
Learning Model**

Today's Session Overview

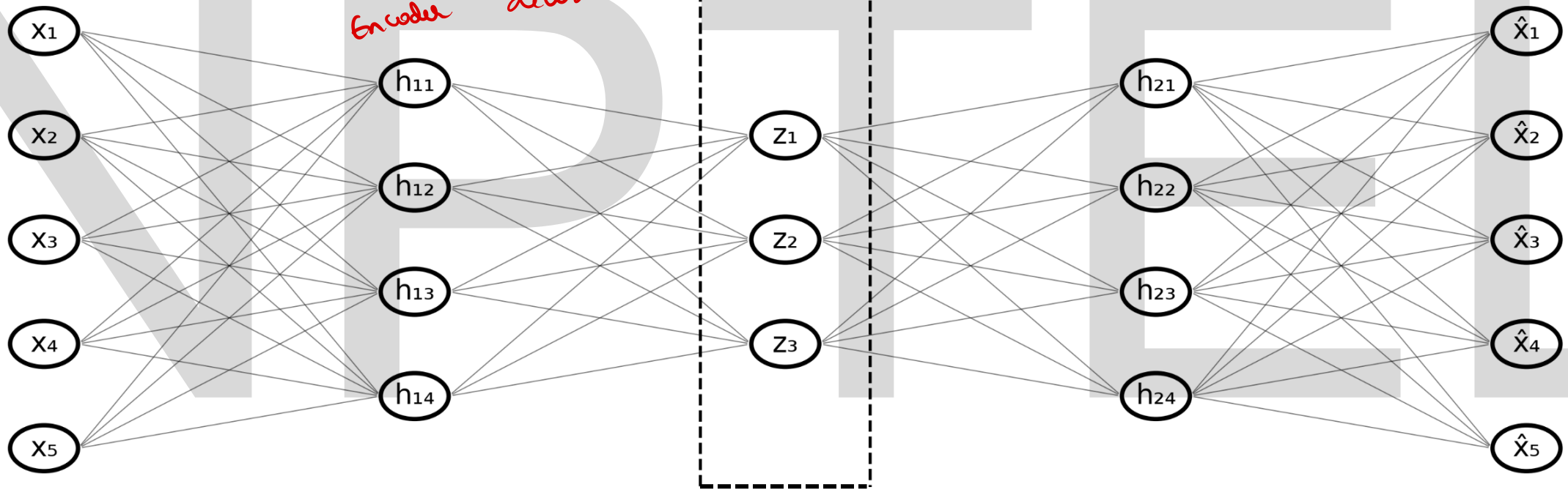
- Step-by-Step Autoencoder Encoding and Reconstruction
- Limitations of Autoencoders

NPTEL

Autoencoder Numerical Example: From Input to Latent Representation to Reconstruction

The architecture:

$5 \rightarrow 4 \rightarrow 3 \rightarrow 4 \rightarrow 5$
Encoder Latent Decoder



The architecture:

$$5 \rightarrow 4 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

Input sample:

$$x = [0.8, 0.4, 0.6, 0.2, 0.5]$$

ReLU activation is used in the hidden layers.

Encoder: Input Layer to Hidden Layer 1

Input:

$$x = [0.8, 0.4, 0.6, 0.2, 0.5]$$

Weight matrix:

$$W_1 = \begin{bmatrix} 0.2 & 0.1 & 0.4 & 0.3 \\ 0.5 & 0.3 & 0.2 & 0.1 \\ 0.4 & 0.2 & 0.1 & 0.5 \\ 0.3 & 0.6 & 0.2 & 0.4 \\ 0.1 & 0.4 & 0.3 & 0.2 \end{bmatrix}$$

Single bias value:

$$b_1 = 0.01$$

$$h_1 = xW_1 + b_1$$

Calculation of First Hidden Neuron

$$h_{11} = (0.8)(0.2) + (0.4)(0.5) + (0.6)(0.4) + (0.2)(0.3) + (0.5)(0.1) + 0.01$$

$$h_{11} = 0.16 + 0.20 + 0.24 + 0.06 + 0.05 + 0.01$$

$$h_{11} = 0.72$$

$$\text{ReLU}(x) = \max(0, x) = \max(0, 0.72) = 0.72$$

Calculation of Second Hidden Neuron

$$h_{12} = (0.8)(0.1) + (0.4)(0.3) + (0.6)(0.2) + (0.2)(0.6) + (0.5)(0.4) + 0.01$$

$$h_{12} = 0.08 + 0.12 + 0.12 + 0.12 + 0.20 + 0.01$$

$$h_{12} = 0.65$$

$$\text{ReLU}(x) = \max(0, x) = \max(0, 0.65) = 0.65$$

Calculation of Third Hidden Neuron

$$h_{13} = (0.8)(0.4) + (0.4)(0.2) + (0.6)(0.1) + (0.2)(0.2) + (0.5)(0.3) + 0.01$$

$$h_{13} = 0.32 + 0.08 + 0.06 + 0.04 + 0.15 + 0.01$$

$$h_{13} = 0.66$$

$$\text{ReLU}(x) = \max(0, x) = \max(0, 0.66) = 0.66$$

Calculation of Fourth Hidden Neuron

$$h_{14} = (0.8)(0.3) + (0.4)(0.1) + (0.6)(0.5) + (0.2)(0.4) + (0.5)(0.2) + 0.01$$

$$h_{14} = 0.24 + 0.04 + 0.30 + 0.08 + 0.10 + 0.01$$

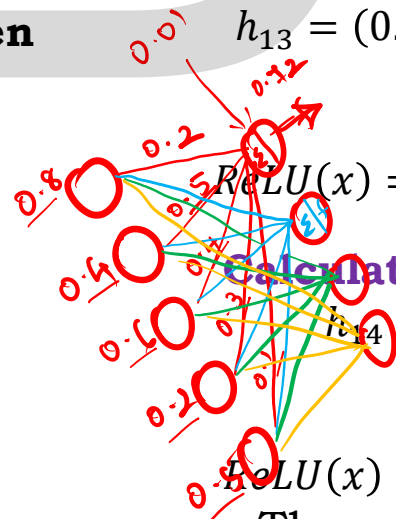
$$h_{14} = 0.77$$

$$\text{ReLU}(x) = \max(0, x) = \max(0, 0.77) = 0.77$$

Therefore,

$$h_1 = [0.72, 0.65, 0.66, 0.77]$$

So, the original 5 values are transformed into 4 hidden values.



Encoder: Hidden Layer 1 to Latent Layer

Now,

$$h_1 = [0.72, 0.65, 0.66, 0.77]$$

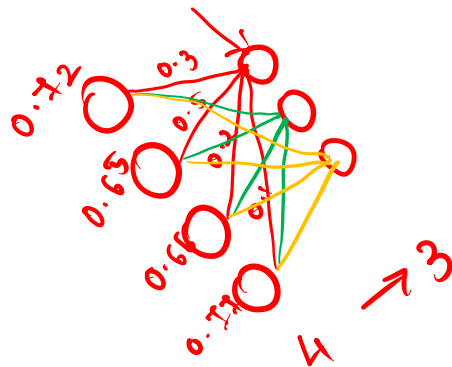
Weight matrix:

$$W_2 = \begin{bmatrix} 0.3 & 0.2 & 0.4 \\ 0.5 & 0.1 & 0.3 \\ 0.2 & 0.4 & 0.1 \\ 0.1 & 0.3 & 0.5 \end{bmatrix}$$

Single bias value:

$$b_2 = 0.01$$

$$z = h_1 W_2 + b_2$$



First Latent Neuron

$$z_1 = (0.72)(0.3) + (0.65)(0.5) + (0.66)(0.2) + (0.77)(0.1) + 0.01$$

$$z_1 = 0.216 + 0.325 + 0.132 + 0.077 + 0.01$$

$$z_1 = 0.760$$

$$\text{ReLU}(x) = \max(0, x) = \max(0, 0.760) = 0.760$$

Second Latent Neuron

$$z_2 = (0.72)(0.2) + (0.65)(0.1) + (0.66)(0.4) + (0.77)(0.3) + 0.01$$

$$z_2 = 0.144 + 0.065 + 0.264 + 0.231 + 0.01$$

$$z_2 = 0.714$$

$$\text{ReLU}(x) = \max(0, x) = \max(0, 0.714) = 0.714$$

Third Latent Neuron

$$z_3 = (0.72)(0.4) + (0.65)(0.3) + (0.66)(0.1) + (0.77)(0.5) + 0.01$$

$$z_3 = 0.288 + 0.195 + 0.066 + 0.385 + 0.01$$

$$z_3 = 0.944$$

$$\text{ReLU}(x) = \max(0, x) = \max(0, 0.944) = 0.944$$

5 → 4 → 3

Therefore, the latent representation is:

$$z = [0.760, 0.714, 0.944]$$

This means the 5-dimensional input is compressed into 3 values:

$$[0.8, 0.4, 0.6, 0.2, 0.5] \rightarrow [0.760, 0.714, 0.944] \leftarrow \text{latent}$$

5 → 3

Decoder: Latent Layer to Decoder Hidden Layer

Now the decoder starts from:

$$z = [0.760, 0.714, 0.944]$$

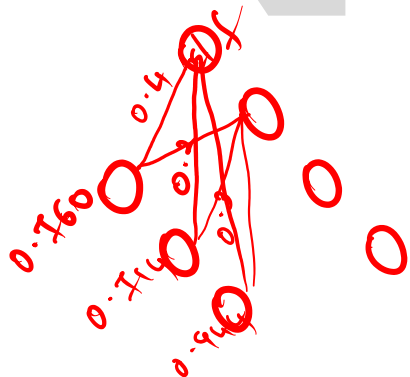
Weight matrix:

$$W_3 = \begin{bmatrix} 0.4 & 0.1 & 0.3 & 0.2 \\ 0.2 & 0.5 & 0.1 & 0.4 \\ 0.3 & 0.2 & 0.4 & 0.1 \end{bmatrix}$$

Single bias value:

$$b_3 = 0.01$$

$$h_2 = zW_3 + b_3$$



First Decoder Hidden Neuron

$$h_{21} = (0.760)(0.4) + (0.714)(0.2) + (0.944)(0.3) + 0.01$$

$$h_{21} = 0.304 + 0.1428 + 0.2832 + 0.01$$

$$h_{21} = 0.740$$

$$ReLU(x) = \max(0, x) = \max(0, 0.740) = 0.740$$

Second Decoder Hidden Neuron

$$h_{22} = (0.760)(0.1) + (0.714)(0.5) + (0.944)(0.2) + 0.01$$

$$h_{22} = 0.076 + 0.357 + 0.1888 + 0.01$$

$$h_{22} = 0.6318$$

$$ReLU(x) = \max(0, x) = \max(0, 0.6318) = 0.6318$$

Third Decoder Hidden Neuron

$$h_{23} = (0.760)(0.3) + (0.714)(0.1) + (0.944)(0.4) + 0.01$$

$$h_{23} = 0.228 + 0.0714 + 0.3776 + 0.01$$

$$h_{23} = 0.687$$

$$ReLU(x) = \max(0, x) = \max(0, 0.687) = 0.687$$

Fourth Decoder Hidden Neuron

$$h_{24} = (0.760)(0.2) + (0.714)(0.4) + (0.944)(0.1) + 0.01$$

$$h_{24} = 0.152 + 0.2856 + 0.0944 + 0.01$$

$$h_{24} = 0.542$$

$$ReLU(x) = \max(0, x) = \max(0, 0.542) = 0.542$$

Therefore,

$$h_2 = [0.740, 0.6318, 0.687, 0.542]$$

Decoder: Decoder Hidden Layer to Output Layer

Now,

$$h_2 = [0.740, 0.6318, 0.687, 0.542]$$

Weight matrix:

$$W_4 = \begin{bmatrix} 0.3 & 0.2 & 0.4 & 0.1 & 0.2 \\ 0.1 & 0.5 & 0.2 & 0.3 & 0.4 \\ 0.4 & 0.1 & 0.3 & 0.2 & 0.1 \\ 0.2 & 0.3 & 0.1 & 0.5 & 0.3 \end{bmatrix}$$

Single bias value:

$$b_4 = 0.01$$

$$\hat{x} = h_2 W_4 + b_4$$

Since the input features are continuous, the reconstructed output must also be continuous. Hence, a linear activation is used at the output layer:

$$f(a) = a$$

This means the output neuron returns the computed value directly.

First Reconstructed Feature

$$\hat{x}_1 = (0.740)(0.3) + (0.6318)(0.1) + (0.687)(0.4) + (0.542)(0.2) + 0.01$$

$$\hat{x}_1 = 0.222 + 0.06318 + 0.2748 + 0.1084 + 0.01$$

$$\hat{x}_1 = 0.67838$$

$f(x) = x$
 $a = a$
 $= 0.67838$

Second Reconstructed Feature

$$\hat{x}_2 = (0.740)(0.2) + (0.6318)(0.5) + (0.687)(0.1) + (0.542)(0.3) + 0.01$$

$$\hat{x}_2 = 0.148 + 0.3159 + 0.0687 + 0.1626 + 0.01$$

$$\hat{x}_2 = 0.7052$$

Third Reconstructed Feature

$$\hat{x}_3 = (0.740)(0.4) + (0.6318)(0.2) + (0.687)(0.3) + (0.542)(0.1) + 0.01$$

$$\hat{x}_3 = 0.296 + 0.12636 + 0.2061 + 0.0542 + 0.01$$

$$\hat{x}_3 = 0.69266$$

Fourth Reconstructed Feature

$$\hat{x}_4 = (0.740)(0.1) + (0.6318)(0.3) + (0.687)(0.2) + (0.542)(0.5) + 0.01$$

$$\hat{x}_4 = 0.074 + 0.18954 + 0.1374 + 0.271 + 0.01$$

$$\hat{x}_4 = 0.68194$$

Fifth Reconstructed Feature

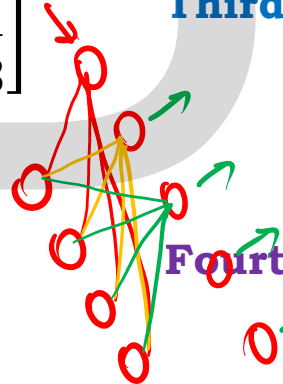
$$\hat{x}_5 = (0.740)(0.2) + (0.6318)(0.4) + (0.687)(0.1) + (0.542)(0.3) + 0.01$$

$$\hat{x}_5 = 0.148 + 0.25272 + 0.0687 + 0.1626 + 0.01$$

$$\hat{x}_5 = 0.64202$$

Therefore, the reconstructed output is:

$$\hat{x} = [0.67838, 0.7052, 0.69266, 0.68194, 0.64202]$$



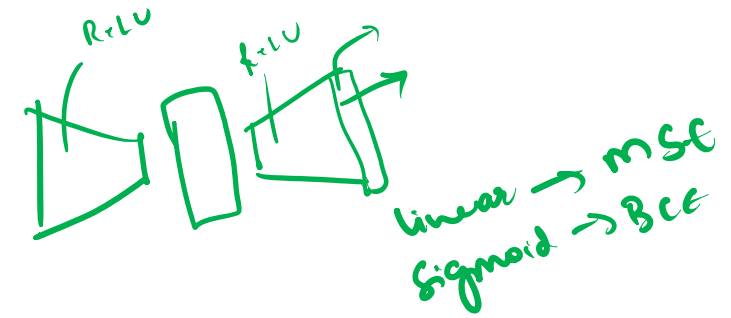
Compare Input and Reconstructed Output

Original input:

$$x = [0.8, 0.4, 0.6, 0.2, 0.5] \checkmark$$

Reconstructed output:

$$\hat{x} = [0.67838, 0.7052, 0.69266, 0.68194, 0.64202]$$



Reconstruction Loss

Since the features are continuous values, we use **Mean Squared Error**.

$$MSE = \frac{1}{5} \sum_{i=1}^5 (x_i - \hat{x}_i)^2$$

$$MSE = \frac{1}{5} [(0.8 - 0.67838)^2 + (0.4 - 0.7052)^2 + (0.6 - 0.69266)^2 + (0.2 - 0.68194)^2 + (0.5 - 0.64202)^2]$$

$$MSE = \frac{1}{5} [0.01479 + 0.09315 + 0.00859 + 0.23227 + 0.02017]$$

$$MSE = \frac{0.36896}{5} = \underline{0.07379}$$

w^* b^*

So, during training, the autoencoder tries to adjust all weights and biases such that the reconstructed output becomes closer and closer to the original input.

$$\underline{x \cdot w^*} \rightarrow \underline{h^*} \rightarrow \underline{h^* w^{*T}} \rightarrow \underline{\hat{x}}$$

Limitations of Autoencoders

1. An autoencoder may reconstruct the input with low reconstruction error,

$$\mathcal{L}_{AE} = \|x - \hat{x}\|^2$$

a low reconstruction error does not always mean that the model has learned meaningful features.—i.e., in handwritten digit images, the model may reconstruct the digit well, but the latent variables may not clearly represent:

whether the digit is 2 or 9, whether the stroke is thick or thin, or whether the digit is slanted.

2. This compressed information is not guaranteed to be useful for tasks such as classification, prediction, clustering, or anomaly detection, even if the reconstruction error is low.

3. Since the reconstructed output is generated from a compressed latent representation, the decoder produces only an approximation of the input, the model cannot generate new samples.



Limitation 4: Choosing the Latent Dimension is Critical

The size of the latent vector strongly affects the quality of representation.

Case 1: Latent space too small

If the latent dimension is too small, the model is forced to compress the input excessively.

This may cause:

information loss → poor reconstruction

The reconstructed output may become blurry, incomplete, or distorted.

Case 2: Latent space too large

If the latent dimension is too large, the model may simply memorize the input.

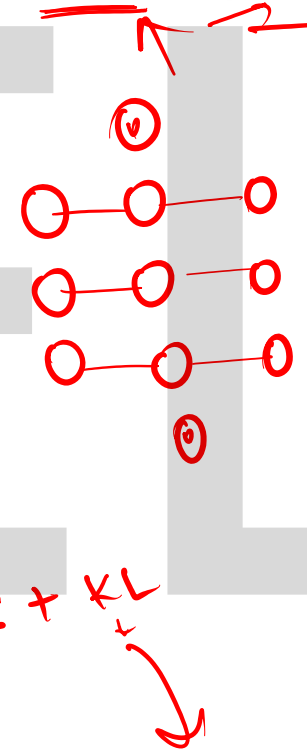
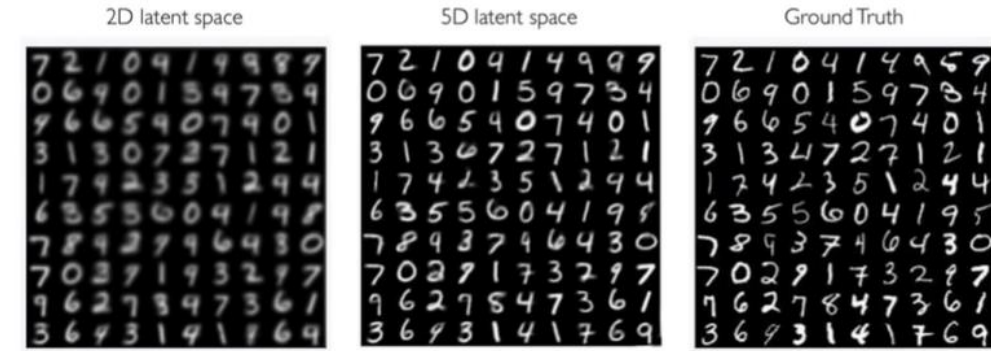
This may cause:

low reconstruction error → weak feature learning

The model reconstructs well but fails to learn compact and useful representations.

Variational Autoencoders (VAEs) address these limitations by introducing regularization on the latent space.

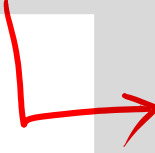
This is achieved by adding a **KL divergence regularization term** to the **loss function**.



Next Session

- Hands-on Implementation of types of AEs

NPTEL



NIPTELL

Thank you